

<u>Object Oriented Programming</u> <u>Assignment 3</u>	<u>Instructor: Mam Hina Iqbal</u> <u>TA: Taimoor Shakeel (1226822@lhr.nu.edu.pk)</u>	<u>Section : Bcs-2F</u> <u>Due Date:12/04/2026</u>
---	---	---

Question: Zoo Management System

Marks: 20 | Concepts: Aggregation & Composition

Problem Overview

You are required to build Zuces a console-based zoo simulation in C++ that demonstrates both Composition and Aggregation relationships across multiple interconnected classes.

Relationship Map

Relationship	Parent -> Child	Child Lifetime	Type
Zoo -> Enclosure	Enclosures live inside Zoo	Destroyed with Zoo	Composition
Enclosure -> Animal	Animals live inside Enclosure	Destroyed with Enclosure	Composition
Zookeeper -> Address	Address lives inside Zookeeper	Destroyed with Zookeeper	Composition
Visitor -> Ticket	Ticket lives inside Visitor	Destroyed with Visitor	Composition
Zoo -> Visitor	Zoo holds pointer only	Visitor survives Zoo	Aggregation
Zoo -> Zookeeper	Zoo holds pointer only	Zookeeper survives Zoo	Aggregation
Visitor -> Animal	Visitor holds pointer only	Animal survives Visitor	Aggregation

Class Definitions

Class 1 - Address (Composition inside Zookeeper)

```
class Address {
private:
    char* street;
    char* city;
    char* country;
public:
    Address (const char* street, const char* city, const char* country);
    Address (const Address& other);
    Address& operator=(const Address& other);
    ~Address();
    void display() const;
};
```

Sample Output: Address: 12-Garden Lane, Lahore, Pakistan

Class 2 - Ticket (Composition inside Visitor)

```
class Ticket {
private:
    char* ticketId;
    char* type; // e.g., Adult, Child
public:
    Ticket (const char* id, const char* type);
    ~Ticket();
    void display() const;
};
```

Sample Output: Ticket: T-101 | Type: Adult

Class 3 - Animal (Composition inside Enclosure)

```
class Animal {
private:
    char* species;
    char* name;
    int age;
public:
    Animal (const char* species, const char* name, int age);
    ~Animal();
    void display() const;
};
```

Sample Output: Animal: Lion (Name: Simba, Age: 5)

Class 4 - Enclosure (Composition inside Zoo)

```
class Enclosure {
private:
    char* enclosureName;
    Animal* animals;
    int animalCount;
public:
    Enclosure (const char* name);
    ~Enclosure(); // destroys animals
    void addAnimal (const char* species, const char* name, int age);
    void display() const;
};
```

Class 5 - Zookeeper (Independent Entity)

```
class Zookeeper {
private:
```

```

    char* name;
    char* employeeId;
    Address address; // composition
public:
    Zookeeper (const char* name, const char* id, const Address& addr);
    ~Zookeeper();
    void display() const;
};

```

Class 6 - Visitor (Independent Entity)

```

class Visitor {
private:
    char* name;
    Ticket ticket; // composition
    Animal ** visitedAnimals; // aggregation
    int visitCount;
public:
    Visitor (const char* name, const Ticket& t);
    ~Visitor(); // does NOT delete animals
    void visitAnimal (Animal* a);
    void leaveAnimal (const char* animalName);
    void display() const;
};

```

Class 7 - Zoo (Top-Level Class)

```

class Zoo {
private:
    char* zooName;
    Enclosure* enclosures; // composition
    int enclosureCount;
    Visitor ** visitors; // aggregation
    int visitorCount;
    Zookeeper ** keepers; // aggregation
    int keeperCount;
public:
    Zoo (const char* name);
    ~Zoo(); // destroys enclosures, not visitors or keepers
    void addEnclosure (const char* name);
    void registerVisitor (Visitor* v);
    void assignZookeeper (Zookeeper* k);
    void display() const;
};

```

What main() Must Demonstrate

- Create Zookeepers Independently (proving they exist outside Zoo).
- Build Zoo and Add Enclosures (Composition).
- Add Animals to Enclosures (Composition).
- Create Visitors and Register Them with Zoo (Aggregation).
- Visitors Visit Animals (Aggregation).
- Duplicate Assignment Edge Case (Visitor already registered).
- Drop Visit Edge Case (Animal not found in visitor's list).
- Destroy Zoo -> Enclosures and Animals destroyed, but Visitors and Zookeepers survive.
- Destroy Visitor -> Ticket destroyed with Visitor, but Animals survive.

Edge Cases (Must Handle)

- Adding duplicate visitor to Zoo -> print error.
- Visitor visiting same animal twice -> print error.
- Removing non-existent visitor from Zoo -> print error.
- Accessing non-existent enclosure/animal -> return nullptr.

Sample Output

```
Zookeeper created: Ali Khan [ID: ZK-101]  
Address: 12-Garden Lane, Lahore, Pakistan
```

```
Zoo: Lahore Safari Park  
Enclosure: Big Cats  
Animal: Lion (Simba, Age: 5)
```

```
Visitor created: Sara Ahmed  
Ticket: T-101 | Type: Adult  
Visitor Sara visits Simba the Lion
```

```
Destroying Zoo Lahore Safari Park ...  
Enclosure Big Cats destroyed. Animals gone.  
Visitor Sara Ahmed still exists -> Aggregation confirmed.
```